

PCCA2 Project Report

Real Root Isolation

Encadrant: Edern Gillot

Hongjing ZHU

Sorbonne Université
Master Informatique M1 – CCA

1 Introduction

The problem of computing and isolating the real roots of a univariate polynomial is a fundamental question in computer algebra. Given a polynomial with real or rational coefficients, one aims to determine all its real roots and to enclose each of them in a disjoint interval with rational endpoints. This process is known as *real root isolation*.

Real root isolation plays an important role in various domains such as symbolic computation, robotics, control theory, and computational geometry. Many applications require not only approximating roots but also guaranteeing their existence and uniqueness within certified intervals.

In this project, we study and implement two classical approaches based on Sturm theory: the *Sturm sequence* and the *Sturm–Habicht sequence*. These methods allow us to count the number of real roots of a polynomial within a given interval by analyzing sign variations.

The main objectives of this project are:

- to implement real root isolation algorithms in C;
- to compare a naive implementation based on Sturm sequences with a more advanced approach using Sturm–Habicht sequences.

2 Theoretical Background

2.1 Real Root Isolation

Let $P(x) \in \mathbb{Q}[x]$ be a univariate polynomial. The problem of real root isolation consists in computing a finite set of pairwise disjoint intervals with rational endpoints such that each interval contains exactly one real root of P , and every real root of P is contained in one of these intervals.

To perform such a task, it is necessary to determine the number of real roots of P in a given interval. This can be achieved using Sturm theory.

2.2 Sturm Sequences

Let $P \in \mathbb{R}[x]$ be a polynomial. The Sturm sequence associated with P is defined as follows:

- $S_0 = P$,
- $S_1 = P'$ (the derivative of P),
- for $i \geq 1$, $S_{i+1} = -\text{rem}(S_{i-1}, S_i)$,

where rem denotes the remainder of the Euclidean division.

The sequence terminates when a constant polynomial is reached.

2.3 Sturm Theorem

Let $a < b$ be two real numbers such that $P(a) \neq 0$ and $P(b) \neq 0$. Let $V(x)$ denote the number of sign variations in the sequence $(S_0(x), S_1(x), \dots)$.

Then, the number of distinct real roots of P in the interval (a, b) is given by:

$$V(a) - V(b).$$

This result allows us to count the number of real roots in any interval.

2.4 Cauchy Bound

To apply Sturm's theorem effectively, it is necessary to restrict the search to a finite interval containing all real roots. The Cauchy bound provides such an interval.

Let

$$P(x) = a_n x^n + a_{n-1} x^{n-1} + \dots + a_0,$$

with $a_n \neq 0$. Then all complex roots z of P satisfy:

$$|z| \leq 1 + \max_{0 \leq i < n} \left| \frac{a_i}{a_n} \right|.$$

In particular, all real roots lie in the interval $[-R, R]$, where R is given by the above bound.

2.5 Sturm–Habicht Sequences

The Sturm–Habicht sequence is a generalization of the Sturm sequence based on subresultant theory. Instead of using classical polynomial remainders, it relies on subresultant polynomials derived from the Sylvester matrix.

This approach has several advantages:

- it provides better numerical stability;
- it avoids coefficient growth in certain cases;
- it is more suitable for exact computations.

Like Sturm sequences, Sturm–Habicht sequences can be used to count real roots by analyzing sign variations.

3 Algorithms

3.1 Root Isolation Using Sturm Sequences

The algorithm based on Sturm sequences proceeds as follows:

1. Compute a bound R using the Cauchy bound such that all real roots lie in $[-R, R]$.

2. Construct the Sturm sequence associated with P .
3. Compute the number of real roots in $[-R, R]$ using Sturm's theorem.
4. Recursively subdivide the interval:
 - for an interval $[a, b]$, compute the number of roots in it;
 - if this number is 0, discard the interval;
 - if it is 1, keep the interval as an isolating interval;
 - otherwise, split the interval into two subintervals and repeat.

This process terminates when all intervals contain exactly one root.

3.2 Root Isolation Using Sturm–Habicht Sequences

The second algorithm follows the same general structure but replaces the Sturm sequence with the Sturm–Habicht sequence.

1. Compute the Sturm–Habicht sequence using subresultant techniques.
2. Evaluate sign variations at interval endpoints.
3. Use these variations to count roots in subintervals.
4. Apply the same recursive subdivision strategy as in the Sturm case.

The main difference lies in the construction of the sequence, which relies on subresultants instead of classical remainders.

3.3 Comparison of the Two Approaches

Both methods rely on the same principle: counting roots via sign variations. However, they differ in their construction and computational properties.

The Sturm sequence is simple to implement and relies on Euclidean division. In contrast, the Sturm–Habicht sequence is more sophisticated and relies on algebraic constructions such as subresultants and determinants.

The Sturm–Habicht approach is expected to improve the execution speed for computations, while the naive Sturm method is easier to implement but slow to execute.

4 Implementation Details

The project is implemented in C and relies on exact rational arithmetic. In order to avoid numerical errors during polynomial manipulations, coefficients are represented with the GMP type `mpq_t`. For some operations involving polynomial evaluation and determinants, the implementation also uses the FLINT library, in particular the types `fmpq_poly_t` and `fmpq_mat_t`.

4.1 General Data Structures

The main data structure is the `Polynomial` structure defined in `include/pcca2.h`. A polynomial is represented by:

- an array `coeffs` of rational coefficients, where `coeffs[i]` stores the coefficient of x^i ;
- an integer `degree`;
- an optional array `roots`, used in the test phase to store known roots for verification.

4.2 Memory Management and Auxiliary Functions

The file `src/util.c` contains the auxiliary routines used throughout the project. The function `create_polynomial` allocates a polynomial of a given degree and initializes all coefficients to zero. The function `free_polynomial` releases both the coefficients and the optional stored roots. A deep copy operation is provided by `copy_polynomial`.

The same file also includes `generate_random_polynomial`, which constructs a polynomial from randomly generated rational roots in a prescribed interval. This is useful for testing the correctness of the root isolation algorithms, since the true roots are known in advance. Finally, the function `afficher_bound` prints the isolated intervals returned by the algorithms.

4.3 Basic Polynomial Operations

The basic algebraic operations are implemented in `src/poly.c`. The derivative is computed by `poly_derivative`. Polynomial multiplication is implemented by the naive `poly_mul_naive`. The Euclidean remainder is computed by `poly_remainder`, which is one of the key ingredients of the naive Sturm sequence.

Several utility operations are also implemented:

- `poly_trim_degree`, which removes trailing zero coefficients and updates the degree;
- `poly_negative`, which multiplies a polynomial by -1 ;
- `poly_get_lc`, which extracts the leading coefficient;
- `poly_is_zero` and `poly_is_monic`, which test structural properties of a polynomial;
- `poly_scalar_mul` and `poly_scalar_div`, which scale all coefficients by a rational factor.

Polynomial evaluation is implemented in two ways. The function `poly_calculate_sign` uses Horner’s method directly on GMP rationals. In addition, the pair `polynomialConv` and `poly_sign_flint` converts the polynomial to FLINT format and evaluates it using FLINT routines. In the recursive isolation step, the implementation uses the FLINT-based evaluation in order to compute sign variations efficiently.

4.4 Root Bounds and Sign Variations

The file `src/calculate.c` gathers the elementary numerical routines needed by both isolation methods.

The function `cauchy_bound` computes the classical Cauchy bound

$$R = 1 + \max_{0 \leq i < n} \left| \frac{a_i}{a_n} \right|,$$

which guarantees that all real roots lie in the interval $[-R, R]$. This interval is used as the starting point for the recursive subdivision procedure.

The function `nb_sign_change` computes the number of sign changes in a sequence while ignoring zeros, which is exactly the quantity needed in Sturm’s theorem. The file also contains `verify_interval`, a testing function that checks whether every known root of a generated polynomial belongs to one of the isolated intervals. Finally, the helper `delta_sign` computes the factor $(-1)^{k(k+1)/2}$ used in the construction of the Sturm–Habicht sequence.

4.5 Naive Sturm Sequence Implementation

The naive implementation of Sturm’s method is contained in `src/sturmNaif.c`. The function `sturm_naif` first constructs the Sturm sequence:

$$S_0 = P, \quad S_1 = P', \quad S_{i+1} = -\text{rem}(S_{i-1}, S_i).$$

The construction stops when a constant polynomial is reached.

Once the sequence has been built, the algorithm computes a global search interval using the Cauchy bound and calls the recursive function `bound_recu`. This function evaluates the whole Sturm sequence at the endpoints of a subinterval, computes the number of sign variations at both ends, and deduces the number of roots in the interval from their difference.

The recursive strategy is the following:

- if an interval contains no root, it is discarded;
- if it contains exactly one root, it is returned as an isolating interval;
- otherwise, the interval is subdivided into smaller subintervals and the process is applied recursively.

The output format is an array of rational values: the first entry stores the number of isolated real roots, and the remaining entries store the interval endpoints.

4.6 Subresultants and Sylvester Matrices

The file `src/subresultant.c` implements the subresultant machinery needed for the Sturm–Habicht approach. First, the code allocates rational matrices with `alloc_matrix`. The function `create_sylv_j_matrix` constructs the truncated Sylvester matrix associated with two polynomials P and S and an index j .

The determinant computations are performed by FLINT. Since the rest of the project uses GMP rationals, the code provides two conversion functions, `mpq_to_fmpq` and `fmpq_to_mpq`, to move between the two libraries. The function `sylv_j_det` extracts the appropriate square minor and computes its determinant. From these determinants, the function `subresultant_j` reconstructs the j -th subresultant polynomial.

This part of the project is more algebraically involved than the naive Sturm implementation, since it explicitly manipulates Sylvester matrices and determinant-based constructions.

4.7 Sturm–Habicht Sequence Implementation

The file `src/sturmHabicht.c` implements the second root isolation method. The function `create_sturm_habicht` constructs the full Sturm–Habicht sequence of a polynomial. The implementation distinguishes two main cases:

- the case where the polynomial is monic, where the sequence starts from P and the remainder of the division of P' by P ;
- the case where the leading coefficient is not equal to 1, where additional scaling by the leading coefficient is required.

The sequence is completed using subresultants together with the sign factor $(-1)^{k(k+1)/2}$. Once the sequence has been constructed, the function `sturm_habicht` applies the same recursive isolation routine `bound_recu` as in the naive Sturm method.

5 Experimental Protocol and Validation

5.1 Test Instances

In order to validate the implementation, we consider polynomials generated by the function `generate_random_polynomial`. This function constructs a polynomial from randomly generated rational roots within a prescribed interval.

This approach provides a convenient testing framework, since the exact real roots of the polynomial are known in advance. As a result, it becomes possible to directly compare the computed isolating intervals with the true roots.

Each test instance is processed independently by both the naive Sturm method or the Sturm–Habicht method.

5.2 Validation of Isolating Intervals

The correctness of the computed isolating intervals is verified using the function `verify_interval`.

Given a polynomial with known roots, and a list of intervals returned by the isolation algorithm, the validation proceeds as follows:

- for each known root, check whether it belongs to at least one of the computed intervals;
- ensure that each interval contains a root;
- verify that the intervals are disjoint.

This verification step guarantees that the output satisfies the definition of real root isolation: every real root is contained in an interval, and each interval isolates exactly one root.

5.3 Evaluation Criteria

The implemented methods are evaluated according to two main criteria.

Correctness. The primary objective is to ensure that both algorithms correctly isolate all real roots. This is assessed through the validation procedure described above, by comparing the computed intervals with the known roots of the test polynomials.

Execution Time. The computational efficiency of the algorithms is also considered. We compare the relative execution time of the naive Sturm method and the Sturm–Habicht method on the same test instances. This comparison provides insight into the practical cost of the more advanced algebraic constructions used in the Sturm–Habicht approach.